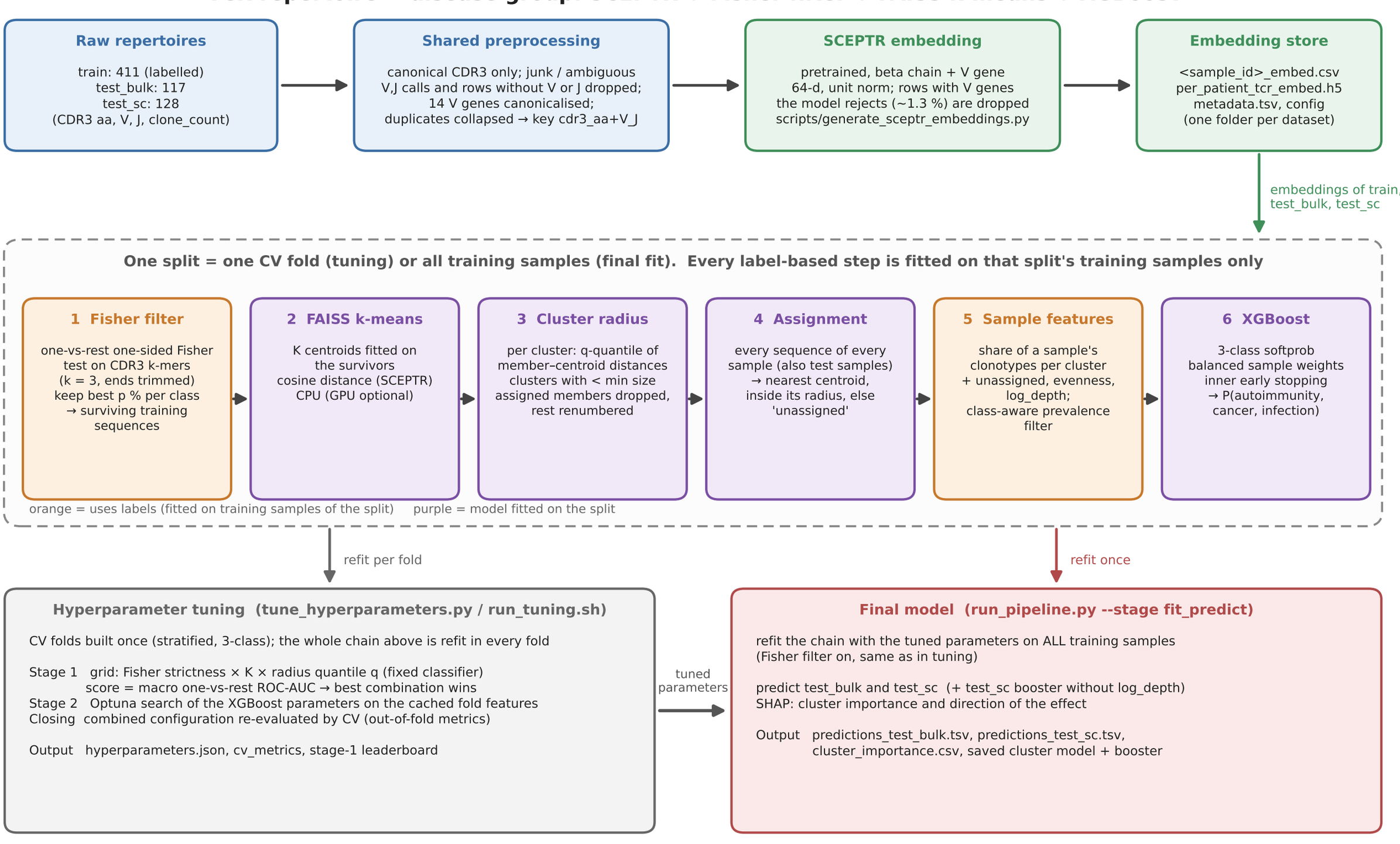
TCR repertoire → disease group: SCEPTR + Fisher filter + FAISS k-means + XGBoost
Raw repertoires
train: 411 (labelled)
test_bulk: 117
test_sc: 128
(CDR3 aa, V, J, clone_count)
Shared preprocessing
canonical CDR3 only; junk / ambiguous
V,J calls and rows without V or J dropped;
14 V genes canonicalised;
duplicates collapsed → key cdr3_aa+V_J
SCEPTR embedding
pretrained, beta chain + V gene
64-d, unit norm; rows with V genes
the model rejects (~1.3 %) are dropped
scripts/generate_sceptr_embeddings.py
Embedding store
<sample_id>_embed.csv
per_patient_tcr_embed.h5
metadata.tsv, config
(one folder per dataset)
embeddings of train,
test_bulk, test_sc
One split = one CV fold (tuning) or all training samples (final fit). Every label-based step is fitted on that split's training samples only
1 Fisher filter
one-vs-rest one-sided Fisher
test on CDR3 k-mers
(k = 3, ends trimmed)
keep best p % per class
→ surviving training
sequences
2 FAISS k-means
K centroids fitted on
the survivors
cosine distance (SCEPTR)
CPU (GPU optional)
3 Cluster radius
per cluster: q-quantile of
member–centroid distances
clusters with < min size
assigned members dropped,
rest renumbered
4 Assignment
every sequence of every
sample (also test samples)
→ nearest centroid,
inside its radius, else
'unassigned'
5 Sample features
share of a sample's
clonotypes per cluster
+ unassigned, evenness,
log_depth;
class-aware prevalence
filter
6 XGBoost
3-class softprob
balanced sample weights
inner early stopping
→ P(autoimmunity,
cancer, infection)
orange = uses labels (fitted on training samples of the split)
purple = model fitted on the split
refit per fold
refit once
Hyperparameter tuning (tune_hyperparameters.py / run_tuning.sh)
CV folds built once (stratified, 3-class); the whole chain above is refit in every fold
Stage 1 grid: Fisher strictness × K × radius quantile q (fixed classifier)
score = macro one-vs-rest ROC-AUC → best combination wins
Stage 2 Optuna search of the XGBoost parameters on the cached fold features
Closing combined configuration re-evaluated by CV (out-of-fold metrics)
Output hyperparameters.json, cv_metrics, stage-1 leaderboard
tuned
parameters
Final model (run_pipeline.py --stage fit_predict)
refit the chain with the tuned parameters on ALL training samples
(Fisher filter on, same as in tuning)
predict test_bulk and test_sc (+ test_sc booster without log_depth)
SHAP: cluster importance and direction of the effect
Output predictions_test_bulk.tsv, predictions_test_sc.tsv,
cluster_importance.csv, saved cluster model + booster